Introduction:

This project studies how Finland's population has changed over time and how it may develop in the near future. I used open population data covering 1990 to 2025 and built a machine learning workflow to estimate future population values for 2030 to 2040.

The main goal was not only to plot the historical trend, but also to prepare the data in a usable format, test a prediction model, and compare growth patterns across different areas.

2. Data Preparation:

I started by loading the raw CSV file and checking its structure. The source table includes area names and yearly population values, so I first renamed the columns into simpler labels.

After that, I filtered the dataset to keep only the end-of-year population measure. I then reshaped the data from wide format into long format. In the long table, each row represents one area in one year.

Finally, I converted the year field into integers and the population values into numeric format. This made the dataset ready for visualization and modeling.

Photos of VS Code:

```
df = df.rename(columns={
    "Alue": "Region",
    "Tiedot": "Indicator"
})

df = df[df["Indicator"] == "Väestö 31.12."].copy()
df.head()
```

[22]

	Region	Indicator	1990	1991	1992	1993	1994	1995	1996	1997	...	2016	2017	2018	2019	2020	2021	2022	2023	2024	2025
0	KOKO MAA	Väestö 31.12.	4998478	5029002	5054982	5077912	5098754	5116826	5132320	5147349	...	5503297	5513130	5517919	5525292	5533793	5548241	5563970	5603851	5635971	5652881
1	Akaa	Väestö 31.12.	16048	16096	16201	16142	16038	15986	15924	15840	...	16923	16769	16611	16475	16391	16467	16473	16405	16387	16429
2	Alajärvi	Väestö 31.12.	11963	11982	12023	11992	11985	11890	11874	11803	...	9899	9831	9700	9562	9419	9311	9183	9113	9078	8982
3	Alavieska	Väestö 31.12.	3072	3060	3053	3097	3084	3051	3054	3064	...	2639	2610	2573	2519	2517	2491	2447	2437	2410	2384
4	Alavus	Väestö 31.12.	13785	13781	13744	13690	13623	13472	13341	13330	...	11907	11713	11544	11468	11332	11197	11102	10933	10780	10634

5 rows × 38 columns

Photos of CSC Supercomputer:

```
[22]: df = df.rename(columns={
    "Alue": "Region",
    "Tiedot": "Indicator"
})

df = df[df["Indicator"] == "Väestö 31.12."].copy()
df.head()
```

[22]:

	Region	Indicator	1990	1991	1992	1993	1994	1995	1996	1997	...	2016	2017	2018	2019	2020	2021	2022	2023	2024	2025
0	KOKO MAA	Väestö 31.12.	4998478	5029002	5054982	5077912	5098754	5116826	5132320	5147349	...	5503297	5513130	5517919	5525292	5533793	5548241	5563970	5603851	5635971	5652881
1	Akaa	Väestö 31.12.	16048	16096	16201	16142	16038	15986	15924	15840	...	16923	16769	16611	16475	16391	16467	16473	16405	16387	16429
2	Alajärvi	Väestö 31.12.	11963	11982	12023	11992	11985	11890	11874	11803	...	9899	9831	9700	9562	9419	9311	9183	9113	9078	8982
3	Alavieska	Väestö 31.12.	3072	3060	3053	3097	3084	3051	3054	3064	...	2639	2610	2573	2519	2517	2491	2447	2437	2410	2384
4	Alavus	Väestö 31.12.	13785	13781	13744	13690	13623	13472	13341	13330	...	11907	11713	11544	11468	11332	11197	11102	10933	10780	10634

5 rows × 38 columns

Photos of VS Code:

```
year_columns = [col for col in df.columns if str(col).isdigit()]

df_long = df.melt(
    id_vars=["Region", "Indicator"],
    value_vars=year_columns,
    var_name="Year",
    value_name="Population"
)

df_long["Year"] = df_long["Year"].astype(int)
df_long["Population"] = pd.to_numeric(df_long["Population"], errors="coerce")

df_long = df_long.sort_values(["Region", "Year"]).reset_index(drop=True)
df_long.head()
```

[23]

	Region	Indicator	Year	Population
0	Akaa	Väestö 31.12.	1990	16048
1	Akaa	Väestö 31.12.	1991	16096
2	Akaa	Väestö 31.12.	1992	16201
3	Akaa	Väestö 31.12.	1993	16142
4	Akaa	Väestö 31.12.	1994	16038

Photos of CSC Supercomputer:

```
[23]: year_columns = [col for col in df.columns if str(col).isdigit()]

df_long = df.melt(
    id_vars=["Region", "Indicator"],
    value_vars=year_columns,
    var_name="Year",
    value_name="Population"
)

df_long["Year"] = df_long["Year"].astype(int)
df_long["Population"] = pd.to_numeric(df_long["Population"], errors="coerce")

df_long = df_long.sort_values(["Region", "Year"]).reset_index(drop=True)
df_long.head()
```

[23]:

	Region	Indicator	Year	Population
0	Akaa	Väestö 31.12.	1990	16048
1	Akaa	Väestö 31.12.	1991	16096
2	Akaa	Väestö 31.12.	1992	16201
3	Akaa	Väestö 31.12.	1993	16142
4	Akaa	Väestö 31.12.	1994	16038

Comparison:

The result I showed here, is from VS Code and CSC supercomputer

But if you check both environments produced the same dataset output, which confirms that the data cleaning and preprocessing steps were performed properly.

And there is no difference in values; only the execution environment is different.

CSC may provide faster performance for large-scale computations.

3. Population Trend Analysis

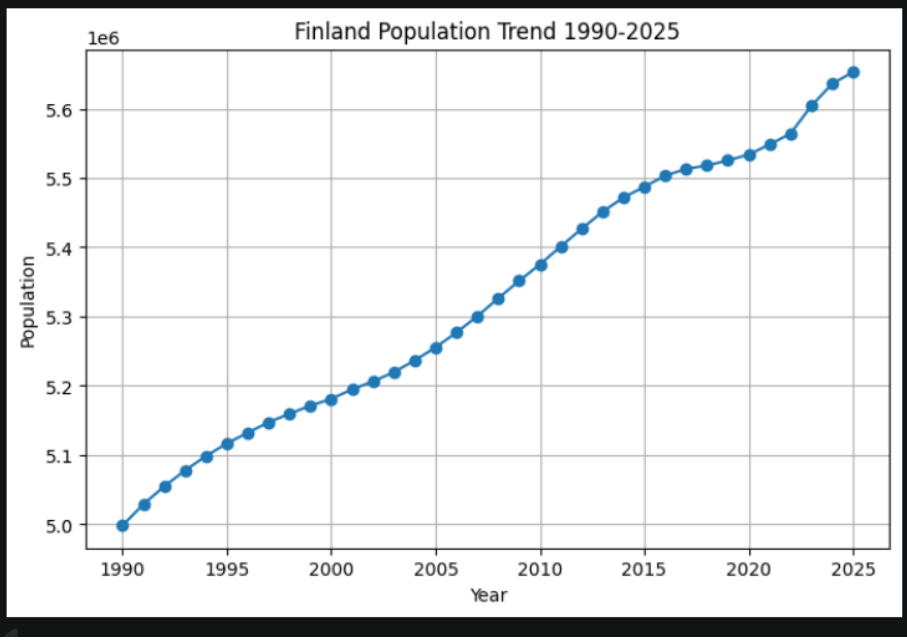
Using the Finland total row, I plotted the population trend from 1990 to 2025. The graph shows a generally upward direction over time, although the speed of growth is not identical every year.

This visualization gives a quick national overview and works as the starting point for the forecasting section.

Photos of VS Code:

```python
plt.figure(figsize=(8, 5))
plt.plot(country["Year"], country["Population"], marker="o")
plt.title("Finland Population Trend 1990-2025")
plt.xlabel("Year")
plt.ylabel("Population")
plt.grid(True)
plt.show()
```

[26]



Photos of Supercomputer:

[6]:
```python
plt.figure(figsize=(8,5))
plt.plot(country["Year"], country["Population"], marker="o")

plt.title("Finland Population Trend 1990-2025")

plt.xlabel("Year")
plt.ylabel("Population")
plt.grid(True)
plt.show()
```

4. Machine Learning Preparation

I have prepared the data for the model by selecting the year as the input and the population as the value to use it for prediction.

After that, I have normalized the data using the average value and standard deviation. This step has helped me to improve the performance of the model and allows it to learn more efficiently.

5. Model Building

I used an MLPRegressor neural network model. The model learns from historical population data and tries to capture the relationship between time and total population.

The purpose of the model is simple: use past observations to estimate future values.

Photos of VS Code:

```
model = MLPRegressor(
    hidden_layer_sizes=(16,),
    activation="relu",
    solver="adam",
    learning_rate_init=0.01,
    max_iter=3000,
    random_state=42
)
model
```

[28]

```
MLPRegressor
▶ Parameters
```

Photos of Supercomputer:

Build TensorFlow regression model: In this step, I have created a neural network model using MLPRegressor:

I have used one hidden layer with 16 neurons Then, I have use ReLU as the activation function Then,I have used Adam as the optimizer Then, I set the learning rate to 0.01 Then, I train the model for a maximum of 3000 iterations Finally, I set a random state to get the same results every time

[8]:
```
from sklearn.neural_network import MLPRegressor

model = MLPRegressor(
    hidden_layer_sizes=(16,),
    activation="relu",
    solver="adam",
    learning_rate_init=0.01,
    max_iter=3000,
    random_state=42
)
```

Train the model :

In this step, I have trained the neural network model:

I have used the scaled input data (X_scaled) and scaled target data (y_scaled)

[9]:
```
model.fit(X_scaled, y_scaled)
```

[9]:
```
MLPRegressor
▶ Parameters
```

Evaluate model on historical data : In this step, I check how well my model is performing:

I use the trained model to make predictions on the training data I calculate the loss using Mean Squared Error (MSE) I also calculate the MAE (Mean Absolute Error)

Results : The loss shows how much error the model has (lower is better) The MAE shows the average difference between predicted and actual values

6. Model Evalution

After training, I used the model to predict the historical data and measured the average error. The loss and MAE values were small, which suggests that the model follows the historical curve reasonably well.

This does not prove perfect forecasting, but it shows that the model has learned the overall pattern in the national series.**Photos of VS Code:**

Results : The loss shows how much error the model has (lower is better) The MAE shows the average difference between predicted and actual values

```
train_preds = model.predict(X_scaled)

loss = np.mean((train_preds - y_scaled) ** 2)
mae = np.mean(np.abs(train_preds - y_scaled))

print("Loss:", loss)
print("MAE (scaled):", mae)
```

✓ 0.0s

```
Loss: 0.01707506806203566
MAE (scaled): 0.1031003282584641
```

Photos of Supercomputer:

Results : The loss shows how much error the model has (lower is better) The MAE shows the average difference between predicted and actual values

0]:
```
train_preds = model.predict(X_scaled)

loss = np.mean((train_preds - y_scaled) ** 2)
mae = np.mean(np.abs(train_preds - y_scaled))

print("Loss:", loss)
print("MAE (scaled):", mae)
```

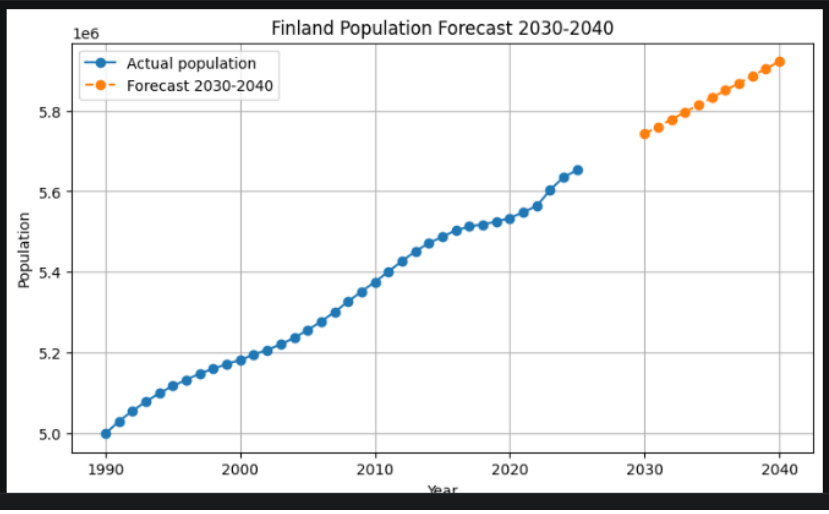
```
Loss: 0.01707506806203566
MAE (scaled): 0.1031003282584641
```

7. Forecast for 2030–2040 :

The trained model was then used to predict Finland's future population for the years 2030 to 2040. The forecast suggests gradual continued growth.

I also plotted the historical values together with the forecast so the transition from observed data to predicted data is easy to understand.
Photos of VS Code:

```python
plt.figure(figsize=(9, 5))

plt.plot(
    country["Year"],
    country["Population"],
    marker="o",
    label="Actual population"
)

plt.plot(
    forecast_df["Year"],
    forecast_df["Predicted Population"],
    marker="o",
    linestyle="--",
    label="Forecast 2030-2040"
)

plt.title("Finland Population Forecast 2030-2040")
plt.xlabel("Year")
plt.ylabel("Population")
plt.legend()
plt.grid(True)
plt.show()
```



```python
future_years = np.arange(2030, 2041)
future_scaled = ((future_years - X_mean) / X_std).reshape(-1, 1)

future_preds_scaled = model.predict(future_scaled)
future_preds = future_preds_scaled * y_std + y_mean

forecast_df = pd.DataFrame({
    "Year": future_years,
    "Predicted Population": future_preds.astype(int)
})

forecast_csv_path = Path("finland_population_forecast_2030_2040.csv")
forecast_df.to_csv(forecast_csv_path, index=False)

print("Saved forecast CSV to:", forecast_csv_path.resolve())
forecast_df
```

```
Saved forecast CSV to: C:\Users\mazum\Downloads\finland_population_forecast_2030_2040.csv
```

	Year	Predicted Population
0	2030	5743041
1	2031	5761029
2	2032	5779017
3	2033	5797004
4	2034	5814992
5	2035	5832980
6	2036	5850968
7	2037	5868956
8	2038	5886943
9	2039	5904931
10	2040	5922919

Photos of Supercomputer :

[11]:
```
future_years = np.arange(2030, 2041)

future_scaled = (future_years - X_mean) / X_std
future_scaled = future_scaled.reshape(-1, 1)

future_preds_scaled = model.predict(future_scaled)

future_preds = future_preds_scaled * y_std + y_mean

forecast_df = pd.DataFrame({
    "Year": future_years,
    "Predicted Population": future_preds.astype(int)
})

forecast_df
```

[11]:

	Year	Predicted Population
0	2030	5743041
1	2031	5761029
2	2032	5779017
3	2033	5797004
4	2034	5814992
5	2035	5832980
6	2036	5850968
7	2037	5868956
8	2038	5886943
9	2039	5904931
10	2040	5922919

[31]:
```
future_years = np.arange(2030, 2041)
future_scaled = ((future_years - X_mean) / X_std).reshape(-1, 1)

future_preds_scaled = model.predict(future_scaled)
future_preds = future_preds_scaled * y_std + y_mean

forecast_df = pd.DataFrame({
    "Year": future_years,
    "Predicted Population": future_preds.astype(int)
})

forecast_csv_path = Path("finland_population_forecast_2030_2040.csv")
forecast_df.to_csv(forecast_csv_path, index=False)

print("Saved forecast CSV to:", forecast_csv_path.resolve())
forecast_df
```

```
Saved forecast CSV to: C:\Users\mazum\Downloads\finland_population_forecast_2030_2040.csv
```

[31]:

	Year	Predicted Population
0	2030	5743041
1	2031	5761029
2	2032	5779017
3	2033	5797004
4	2034	5814992
5	2035	5832980
6	2036	5850968
7	2037	5868956
8	2038	5886943
9	2039	5904931
10	2040	5922919

8. Growth Analysis by Area

To compare areas, I calculated the yearly percentage change in population for each area and then summarized the average growth rate.

This made it possible to identify places with stronger long-term growth and areas with long-term decline. These differences matter because fast-growing places may need more housing and infrastructure, while shrinking areas may face demographic pressure and service challenges.

Photos of VS Code:

```
...  Top growing regions:
     Region
     Liminka                  0.021624
     Kempele                  0.020176
     Jomala                   0.018910
     Espoo                    0.018315
     Pirkkala                 0.018166
     TA01 Espoo-Kauniainen    0.017921
     Lempäälä                 0.015632
     Lemland                  0.015104
     Kirkkonummi              0.014349
     Vantaa                   0.014113
     Name: Growth Rate, dtype: float64

     Most declining regions:
     Region
     Rautjärvi    -0.017756
     Ilomantsi    -0.017783
     Siikainen    -0.018293
     Posio        -0.018693
     Salla        -0.018718
     Rääkkylä     -0.018755
     Puolanka     -0.019439
     Kivijärvi    -0.019529
     Hyrynsalmi   -0.020253
     Rautavaara   -0.021327
     Name: Growth Rate, dtype: float64
```

Photos of Supercomputer:

```
Top growing regions:
Region
Liminka                  0.021624
Kempele                  0.020176
Jomala                   0.018910
Espoo                    0.018315
Pirkkala                 0.018166
TA01 Espoo-Kauniainen    0.017921
Lempäälä                 0.015632
Lemland                  0.015104
Kirkkonummi              0.014349
Vantaa                   0.014113
Name: Growth Rate, dtype: float64

Most declining regions:
Region
Rautjärvi    -0.017756
Ilomantsi    -0.017783
Siikainen    -0.018293
Posio        -0.018693
Salla        -0.018718
Rääkkylä     -0.018755
Puolanka     -0.019439
Kivijärvi    -0.019529
Hyrynsalmi   -0.020253
Rautavaara   -0.021327
Name: Growth Rate, dtype: float64
```

9. Anomaly Detection

I measured year-to-year changes in the growth rate and marked unusual values using a three-standard-deviation rule.

These anomalies can highlight years when population movement was stronger than usual or when a region behaved differently from its normal pattern.
For VS Code :

```python
anomalies = df_long[df_long["Anomaly"]].copy()
anomalies.head(20)
```

	Region	Indicator	Year	Population	Growth Rate	Growth Change	Anomaly
218	Aura	Väestö 31.12.	1992	3195	0.019464	-0.039320	True
258	Brändö	Väestö 31.12.	1996	534	-0.025547	-0.029210	True
259	Brändö	Väestö 31.12.	1997	538	0.007491	0.033038	True
260	Brändö	Väestö 31.12.	1998	525	-0.024164	-0.031654	True
264	Brändö	Väestö 31.12.	2002	502	0.001996	0.027288	True
270	Brändö	Väestö 31.12.	2008	518	0.015686	0.034917	True
271	Brändö	Väestö 31.12.	2009	498	-0.038610	-0.054296	True
279	Brändö	Väestö 31.12.	2017	452	-0.040340	-0.042467	True
280	Brändö	Väestö 31.12.	2018	449	-0.006637	0.033703	True
285	Brändö	Väestö 31.12.	2023	436	-0.031111	-0.033338	True
651	Eckerö	Väestö 31.12.	1993	829	-0.016607	-0.042155	True
656	Eckerö	Väestö 31.12.	1998	835	0.030864	0.038217	True
658	Eckerö	Väestö 31.12.	2000	830	-0.018913	-0.032086	True
659	Eckerö	Väestö 31.12.	2001	839	0.010843	0.029756	True
660	Eckerö	Väestö 31.12.	2002	884	0.053635	0.042792	True
661	Eckerö	Väestö 31.12.	2003	895	0.012443	-0.041192	True
663	Eckerö	Väestö 31.12.	2005	925	0.031215	0.028981	True
670	Eckerö	Väestö 31.12.	2012	960	-0.018405	-0.055520	True
675	Eckerö	Väestö 31.12.	2017	948	0.021552	0.029038	True
679	Eckerö	Väestö 31.12.	2021	933	-0.026096	-0.032399	True

For Supercomputer:

```
[34]: anomalies = df_long[df_long["Anomaly"]].copy()
      anomalies.head(20)
```

[34]:

	Region	Indicator	Year	Population	Growth Rate	Growth Change	Anomaly
218	Aura	Väestö 31.12.	1992	3195	0.019464	-0.039320	True
258	Brändö	Väestö 31.12.	1996	534	-0.025547	-0.029210	True
259	Brändö	Väestö 31.12.	1997	538	0.007491	0.033038	True
260	Brändö	Väestö 31.12.	1998	525	-0.024164	-0.031654	True
264	Brändö	Väestö 31.12.	2002	502	0.001996	0.027288	True
270	Brändö	Väestö 31.12.	2008	518	0.015686	0.034917	True
271	Brändö	Väestö 31.12.	2009	498	-0.038610	-0.054296	True
279	Brändö	Väestö 31.12.	2017	452	-0.040340	-0.042467	True
280	Brändö	Väestö 31.12.	2018	449	-0.006637	0.033703	True
285	Brändö	Väestö 31.12.	2023	436	-0.031111	-0.033338	True
651	Eckerö	Väestö 31.12.	1993	829	-0.016607	-0.042155	True
656	Eckerö	Väestö 31.12.	1998	835	0.030864	0.038217	True
658	Eckerö	Väestö 31.12.	2000	830	-0.018913	-0.032086	True
659	Eckerö	Väestö 31.12.	2001	839	0.010843	0.029756	True
660	Eckerö	Väestö 31.12.	2002	884	0.053635	0.042792	True
661	Eckerö	Väestö 31.12.	2003	895	0.012443	-0.041192	True
663	Eckerö	Väestö 31.12.	2005	925	0.031215	0.028981	True
670	Eckerö	Väestö 31.12.	2012	960	-0.018405	-0.055520	True
675	Eckerö	Väestö 31.12.	2017	948	0.021552	0.029038	True
679	Eckerö	Väestö 31.12.	2021	933	-0.026096	-0.032399	True

10. Clustering Analysis :

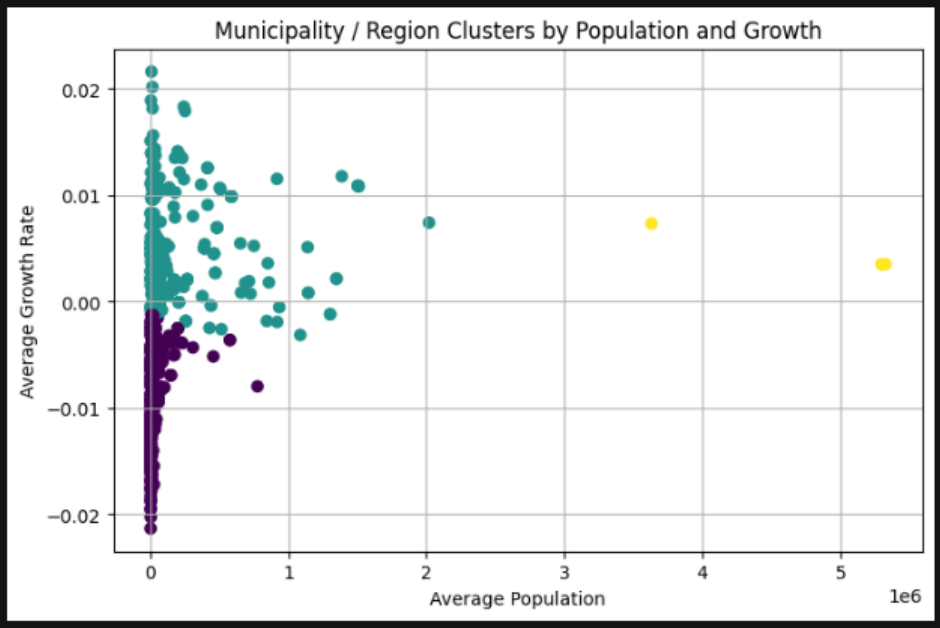
I grouped areas using average population and average growth rate. Before clustering, I scaled the variables so that neither one would dominate the grouping.

Then I applied K-Means with three clusters. The cluster plot helps show which areas are similar in size and growth behavior.
Photos of VS Code:

```python
cluster_data = (
    df_long.groupby("Region")
    .agg(
        avg_population=("Population", "mean"),
        avg_growth=("Growth Rate", "mean")
    )
    .dropna()
)

scaler = StandardScaler()
cluster_scaled = scaler.fit_transform(cluster_data)

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
cluster_data["Cluster"] = kmeans.fit_predict(cluster_scaled)

cluster_data.head()
```

	avg_population	avg_growth	Cluster
Region			
Akaa	16443.722222	0.000686	1
Alajärvi	10670.388889	-0.008143	0
Alavieska	2792.611111	-0.007185	0
Alavus	12500.138889	-0.007377	0
Asikkala	8464.555556	-0.003349	0

```python
plt.figure(figsize=(8, 5))
plt.scatter(
    cluster_data["avg_population"],
    cluster_data["avg_growth"],
    c=cluster_data["Cluster"]
)
plt.title("Municipality / Region Clusters by Population and Growth")
plt.xlabel("Average Population")
plt.ylabel("Average Growth Rate")
plt.grid(True)
plt.show()
```



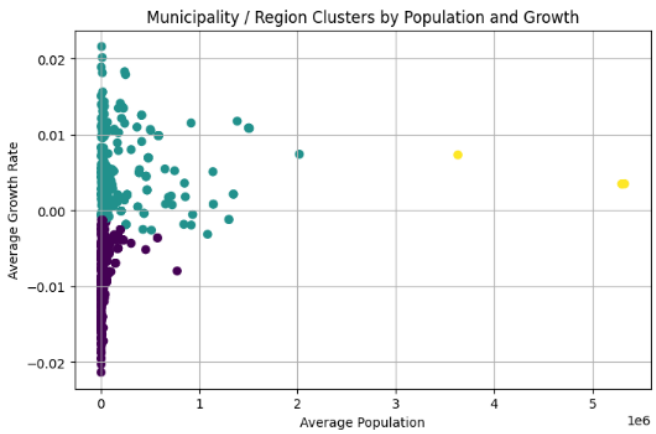
For Supercomputer:

[35]:
```
cluster_data = (
    df_long.groupby("Region")
    .agg(
        avg_population=("Population", "mean"),
        avg_growth=("Growth Rate", "mean")
    )
    .dropna()
)

scaler = StandardScaler()
cluster_scaled = scaler.fit_transform(cluster_data)

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
cluster_data["Cluster"] = kmeans.fit_predict(cluster_scaled)

cluster_data.head()
```

[35]:

Region	avg_population	avg_growth	Cluster
Akaa	16443.722222	0.000686	1
Alajärvi	10670.388889	-0.008143	0
Alavieska	2792.611111	-0.007185	0
Alavus	12500.138889	-0.007377	0
Asikkala	8464.555556	-0.003349	0

[36]:
```
plt.figure(figsize=(8, 5))
plt.scatter(
    cluster_data["avg_population"],
    cluster_data["avg_growth"],
    c=cluster_data["Cluster"]
)
plt.title("Municipality / Region Clusters by Population and Growth")
plt.xlabel("Average Population")
plt.ylabel("Average Growth Rate")
plt.grid(True)
plt.show()
```



11. Comparison: VS Code vs CSC :

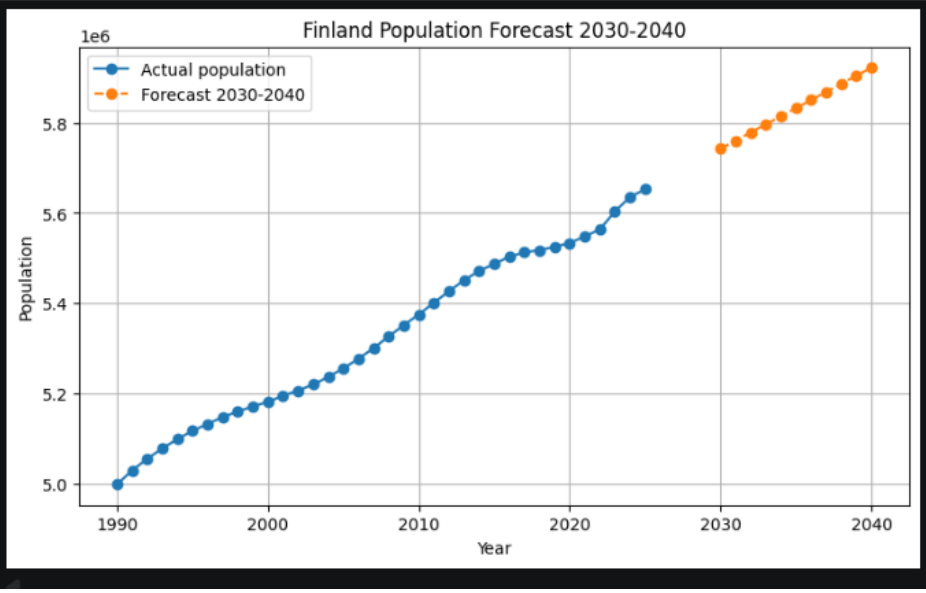
I ran the same type of workflow in both VS Code and CSC-style environments. The numerical outputs should remain very close when the same code, same data, and

same package behavior are used.

The practical difference is mostly in the environment:
- VS Code is easier for writing and editing
- CSC is more useful when computation becomes larger or heavier

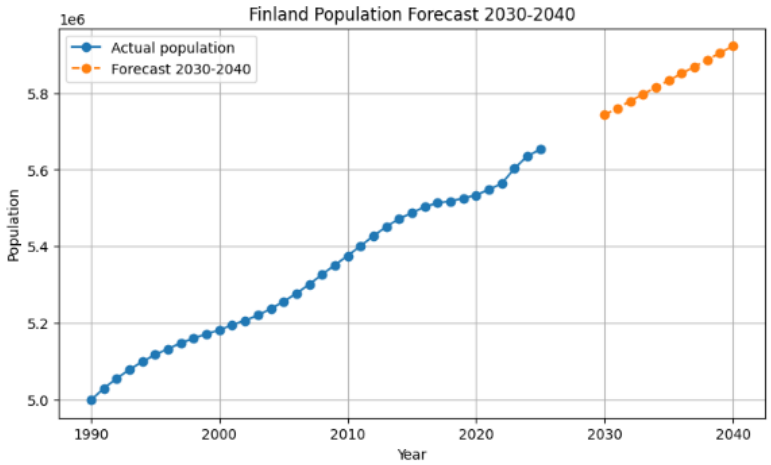
VS Code Forecast graph ;

```
plt.figure(figsize=(9, 5))

plt.plot(
    country["Year"],
    country["Population"],
    marker="o",
    label="Actual population"
)

plt.plot(
    forecast_df["Year"],
    forecast_df["Predicted Population"],
    marker="o",
    linestyle="--",
    label="Forecast 2030-2040"
)

plt.title("Finland Population Forecast 2030-2040")
plt.xlabel("Year")
plt.ylabel("Population")
plt.legend()
plt.grid(True)
plt.show()
```



Supercomputer Forecast Graph :

```
[32]: plt.figure(figsize=(9, 5))

plt.plot(
    country["Year"],
    country["Population"],
    marker="o",
    label="Actual population"
)

plt.plot(
    forecast_df["Year"],
    forecast_df["Predicted Population"],
    marker="o",
    linestyle="--",
    label="Forecast 2030-2040"
)

plt.title("Finland Population Forecast 2030-2040")
plt.xlabel("Year")
plt.ylabel("Population")
plt.legend()
plt.grid(True)
plt.show()
```



VS Code Loss/MAE:

```
train_preds = model.predict(X_scaled)

loss = np.mean((train_preds - y_scaled) ** 2)
mae = np.mean(np.abs(train_preds - y_scaled))

print("Loss:", loss)
print("MAE (scaled):", mae)
[30]

Loss: 0.01707506806203566
MAE (scaled): 0.1031003282584641
```

CSC Loss/MAE :

```
[30]: train_preds = model.predict(X_scaled)

      loss = np.mean((train_preds - y_scaled) ** 2)
      mae = np.mean(np.abs(train_preds - y_scaled))

      print("Loss:", loss)
      print("MAE (scaled):", mae)

      Loss: 0.01707506806203566
      MAE (scaled): 0.1031003282584641
```

Conclusion

Overall, the project shows that Finland's population has generally increased over the observed period, and the model projects further growth into 2040.

At the same time, the regional analysis shows that not all areas move in the same direction. Some places grow faster, some are stable, and others decline. That is why growth analysis, anomaly detection, and clustering are useful additions to the national forecast.